Lecture Note: GitHub Workflow & Collaboration

Nishut Suman 2 Jan, 2026 At 6:00 PM Notes Foundation Recommended Resource

Associated Lectures (1)

Description Discussions

Lecture Notes: GitHub Workflow & Collaboration

Prerequisites: Understanding of basic Git commands (**init, add, commit**), local repositories, and branching.

Familiarity with Session 11 (Version Control Fundamentals).

What you'll be able to do:

- Apply GitHub collaboration workflows using remotes, push, pull, and Pull Requests (PRs)
- Resolve merge conflicts confidently
- Coordinate code reviews and merges with teammates
- Understand how GitHub fits into real-world team development and CI/CD pipelines

---

1. Introduction: What Is GitHub Collaboration and Why Should You Care?

Core Definition

GitHub collaboration refers to the practice of connecting multiple developers to a shared codebase using Git and GitHub. It involves working with **remote repositories**, **branches**, **pull requests**, and **reviews** to integrate changes efficiently and safely.

A Simple Analogy

Think of GitHub as a shared online **workspace** for your code. Each developer works at their own desk (local branch), and when ready, submits their work to the central table (main branch) through a Pull Request for everyone to review.

Limitation: Unlike a physical workspace, GitHub automatically tracks every change, version, and merge - you can roll back any mistake without losing history.

Why This Matters to You

Problem it solves: Working on the same files with multiple people leads to overwriting and chaos. GitHub introduces structure, safety, and transparency.

What you'll gain:

- **Team synchronization:** Share updates instantly without breaking others' work.
- **Review process:** Improve quality via structured peer review before merging.
- **Conflict resolution:** Learn to detect and fix overlapping changes safely.

Real-world context: All major tech companies (like Netflix, Google, and Microsoft) use GitHub or GitLab to manage and review code collaboratively.

---

2. The Foundation: Core Concepts Explained

Concept A: Remotes

Definition: A *remote* is a hosted copy of your local Git repository on a platform like **GitHub**. It allows syncing changes between your local machine and the team's shared repository.

Key characteristics:

- Linked via URL (HTTPS/SSH)
- Identified by names like **origin** or **upstream**
- Enables **push**, **pull**, and **fetch** operations

Example:

```
git remote add origin <https://github.com/username/project.git>
git push –u origin main
```

> Common confusion: Many think the remote auto-updates - it doesn't. You must explicitly push to send and pull to receive changes.

Concept B: Pull Requests (PRs)

Definition: A PR is a GitHub-based mechanism to propose, discuss, and review changes before merging them into the main branch.

Relation to Remotes: You push your branch to the remote, then open a PR - the PR lives on GitHub, but the code lives in your branch.

Key characteristics:

- Supports comments, inline discussions, and CI checks
- Can be merged, closed, or updated via new commits
- Serves as a documentation trail for every feature

Example:

Create a feature branch → **git checkout -b feature/login**

Push it → **git push -u origin feature/login**

Open a PR on GitHub → **feature/login** → **main**

> Remember: PRs promote collaboration over isolation. Every merge should be reviewed.

How Remotes and PRs Work Together

Remotes act as the **bridge**, while PRs are the **gate**. You push to the bridge (remote) and go through the gate (PR review) before your code reaches the main branch.

Visual:

Figure 1: The GitHub Pull Request lifecycle - from feature branch creation to code review and merge.

---

3. Seeing It in Action: Worked Examples

Example 1: Basic Remote Push & Pull

Scenario: You and a teammate are working on the same project. You add a README update locally and want it reflected on GitHub.

Approach:

Connect to a remote

Push local commits

Pull updates from others

Commands:

```
git remote add origin <https://github.com/username/demo.git>
git push -u origin main  # send your work to GitHub
git pull origin main     # bring teammates' updates
```

Output:

```
Enumerating objects...
Writing objects: 100% done
To <https://github.com/username/demo.git>
 * [new branch] main -> main
```

Key idea: GitHub syncs your code between local and remote repositories while preserving history.

Check your understanding: Why do we use **-u** in the first push command?

Example 2: Pull Request with Code Review

Scenario: You've added a new feature on a branch **feature-dark-mode** and want it merged after peer review.

What's different: The change now goes through a review process on GitHub.

Workflow:

```
git checkout -b feature-dark-mode
# Make edits...
git add .
git commit -m "Add dark mode toggle"
git push -u origin feature-dark-mode
```

Then, on GitHub:

Open a PR: **feature-dark-mode → main**

Assign reviewers, discuss, and address feedback

Merge once approved

Visual:

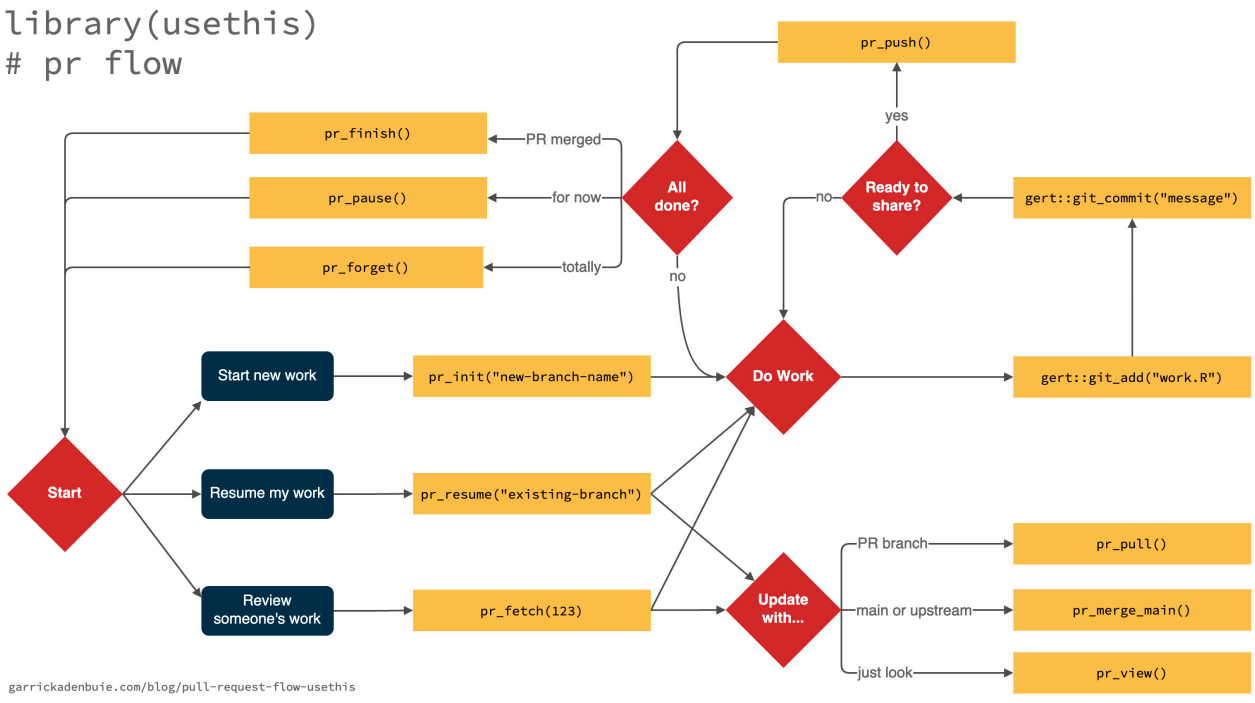

*Figure 2: Diagram showing how a pull request moves through creation, review, and merge stages in GitHub.*Refer image

Key lesson: Code reviews improve quality and create shared ownership of the project.

Example 3: Handling Merge Conflicts

Background: Two developers edited the same line in **index.html**.
Git cannot decide which version to keep.

Conflict appearance:

```
<<<<<<< HEAD
<h1>Welcome to our website!</h1>
=======
<h1>Welcome to my awesome site!</h1>
>>>>>>> origin/main
```

Fix: Choose the correct version, delete the markers, then:

```
git add index.html
git commit –m "Resolve merge conflict"
git push
```

Reference: Merge Conflict Resolution

> Caution: Avoid manual edits without understanding both changes - you might delete important code.

---

4. Common Pitfalls: What Can Go Wrong and How to Avoid It

- **Mistake:** Pushing directly to **main**.
  - **Why it's a problem:** Skips review, introduces bugs.
  - **Right approach:** Always create a feature branch and open a PR.
  - **Why this works:** Encourages peer review and CI validation.
- **Mistake:** Forgetting to pull before pushing.
  - **Why it's a problem:** Leads to rejected pushes or conflicts.
  - **Right approach:** Run **git pull origin main** before your **push**.
  - **Why this works:** Ensures you're merging with the latest code.
- **Mistake:** Merging without resolving all comments.
  - **Why it's a problem:** Leaves incomplete work in **main**.
  - **Right approach:** Mark each comment as resolved and rerun tests.
  - **Why this works:** Maintains code quality and team accountability.

If you're stuck: Revisit the Git basics from Session 11 - especially branching and commit structure.

---

5. Your Turn: Practice & Self-Assessment

Practice Task (Combination: Hands-On + Collaboration)

Challenge: Simulate a real team workflow.

Specifications:

Create a new repository on GitHub (**demo-collaboration**)
Clone it locally and add a teammate as a collaborator
Each member creates a new branch (**feature-A**, **feature-B**)
Both push their changes and open PRs
Merge one PR, then simulate a merge conflict on the second - resolve it

Hint:
Use **git diff** and **git log --oneline --graph** to visualize the history before merging.

Extension: Enable GitHub Actions to auto-run a linter when a PR is opened - this is your first step toward CI automation.

Check Your Understanding

Why do we create Pull Requests instead of pushing directly to main?
What's the difference between a merge conflict and a failed CI check?
How can you keep your branch updated with main without merging manually every time?
What's one best practice you'd apply from this session in your future team project?

Answers:

PRs ensure review, discussion, and safe merges.
Conflicts are code-based; CI failures are validation errors from automated tools.
Use **git fetch origin** and **git rebase origin/main** to sync non-destructively.
Branch-based workflows improve collaboration and prevent code overwrite.

---

6. Consolidation: Key Takeaways & Next Steps

The Essential Ideas

- **Principle:** Collaboration in GitHub revolves around *branches + PRs + review*.
- **Application:** Every feature or bugfix should live on its own branch.
- **Warning:** Never push to **main** without review - it breaks version integrity.

Mental Model Check

Think of GitHub as a **shared journal** - every branch is a new chapter, and pull requests are the editor's review process before publishing.

What You Can Now Do

You can collaborate with others, manage PRs, resolve conflicts, and set the foundation for CI/CD automation. This workflow mirrors what professional teams use daily.

Next Steps

To deepen this knowledge: Learn advanced branching strategies (Git Flow, Trunk-Based Development).
To build on this: Explore **GitHub Actions** to automate tests and deployments.

Additional resources:

- Atlassian Git Tutorials - practical guides on branching and merging.
- GitHub Actions Official Docs - start building CI/CD workflows.

📽️ **Video Reference:** How to Create PR

Quick Reference Card

Command	Purpose
git remote add origin [URL]	Connect local repo to remote
git push origin main	Upload commits to remote
git pull origin main	Fetch and merge latest changes
git checkout -b feature-x	Create a new branch
git merge feature-x	Combine branch into main
git rebase origin/main	Update your branch with latest main

Questions or stuck?

Revisit your GitHub repository's "Insights → Network" tab to visualize branches and merges, or ask teammates for a PR review simulation.

> ✨ "Every professional workflow starts here - the moment you treat your code as a team asset, not a personal one, you step into real software engineering."